



THE UNIVERSITY OF
SYDNEY

Faculty of Engineering

Embedded Systems – ELEC3607

Semester 1 – 2016

Duration: 2 hours and 10 mins reading time

SEAT NUMBER: _____

LAST NAME: _____

FIRST NAME: _____

SID: _____

INSTRUCTIONS TO CANDIDATES

1. Please read each question carefully. The value of each question has been written next to the question. Note that questions are of unequal value. Total mark of this exam paper is 100.
2. This examination consists of four (4) SECTIONS (A, B, C and D).
3. ALL sections must be answered.
4. Write your responses to all questions in the answer booklet provided.
5. Parts of the SAM3X datasheet and an ARM instruction set summary are provided at the end of this paper.
6. Please ensure that your Name and Student No are written on each booklet you use.
7. Please write on the front of each answer booklet the designated Sections (A, B, C and D).
8. Non programmable calculators are permitted
9. Check that you have answered all FOUR (4) sections.

Please check your examination paper is complete (**25 pages**) and indicate you have done this by signing below.

This paper is not to be taken from the examination room.

I have checked the examination paper and affirm it is complete.

Student Name _____ Date: _____

Section A - Peripherals (25 marks)

A data sheet for the SAM3X Serial Peripheral Interface (SPI) peripheral is included at the end of this paper. In answering this question, you must give all constants for register addresses and values in hexadecimal.

1. (10 marks) Assuming that the chip has just been reset, explain the sequence required to initialize SPI1 in master mode. Enable the Variable Peripheral Select mode, such that you can address up to 4 peripheral devices without using a decoder. Configure the protocol mode (clock polarity and phase) for chip select 0 such that an 8-bit data is shifted on the rising edges and captured on the falling edges of the clock. Assuming a MCK of 84 MHz, set the baud rate to 4 Mbps. Do not enable any other mode or option (for example, interrupts or fault detection). The names, addresses and values for all relevant registers should be included and you should justify the values chosen for each field. Do not include any code in your answer.
2. (10 marks) In the C programming language, write a code sequence which will send 0 to 9 as 8-bit values to slave 1 (connected to NPCS1) using SPI1. Use polling, and assume that SPI1 is already configured using your answer in part 1. Do not enable any other mode or option (for example, last transfer signals or shutdown). Your answer should not use any library calls or include files, must be clearly commented and use meaningful variable names.
3. (5 marks) Explain clearly how a UART transmits data, and draw the waveform required to transmit 8-bit value 0x03 with odd parity. Annotate your waveform to show where each bit is transmitted. Label both x and y axis, assuming it is measured from a SAM3X with a baudrate of 1000 bps.

Section B - ARM Assembly Language (25 marks)

An ARM instruction set summary is provided at the end of this paper.

1. (5 marks) In a little-endian system, 32-bit value 0x12345678 is stored in memory location 0x40000000. In a table, with a “memory location” column and a “stored data” column, show what byte values are stored at each memory location.
2. (10 marks) The following is the assembly language translation of a C function.

```
mystery:
    mov    r3, r0
    mov    r0, #0
.L3:
    ldrb   r2, [r3, #1]!
    cmp    r2, r1
    bhi    .L4
    add    r0, r0, #1
    b      .L3
.L4:
    bx     lr
```

Annotate each line of the function with comments to explain what each line does. Write an equivalent C function and explain what it does.

3. (5 marks) Give a single assembly language instruction which will clear bits 4 to 7 of the r2 register.
4. (5 marks) Give a single assembly language instruction which will load the negated value of r2 into r1 only if flag V is set.

Section C - Interrupts (25 marks)

1. (10 marks) Assuming that the chip has just been reset, explain how to program the registers of SPI1, so that it is set to slave mode and an interrupt is generated after each byte of data is received. Assume all clock settings to be similar to Section A. The names, addresses and values for all relevant registers should be included and you should justify the values chosen for each field. Do not include any code in your answer.
2. (10 marks) In the C programming language, write an interrupt handler `spi_irq()` that is called when an SPI receive event occurs. If data has arrived in the receive data register and its value is less than 96, then set Digital Pin 4 to 1, otherwise to 0. If an OVERRUN error has occurred, then set Digital Pin 5 to 1, otherwise to 0. Your answer should not use any library calls or include files (except for `digitalWrite(pin, value)`), must be clearly commented and use CMSIS naming scheme for all registers and flags. You can assume that SPI and other parts of the system, such as NVIC and PIO, have been already programmed.
3. (5 marks) In the context of SPI to UART data transfer (as described in part 2), explain the purpose of using direct memory access. Give an example of a situation in which it is not advantageous over the implementation of part 2.

Section D - Hardware Design (25 marks)

1. (5 marks) Suppose we need to drive 48V DC motor using a 3.3V microcontroller. Draw a circuit which is optically isolated and makes this conversion.
2. (5 marks) Explain one advantage and one disadvantage of a Schottky diode over a traditional P-N junction diode.
3. (5 marks) Name the signals used in I2C busses. Draw an open-drain output and explain why it is used in I2C.
4. (5 marks) Draw a Pierce crystal oscillator. Name each component on your figure.
5. (5 marks) Explain what voltage sag in digital circuits is. Why a network of capacitors is required to reduce this effect instead of only one capacitor?

32. Serial Peripheral Interface (SPI)

32.1 Description

The Serial Peripheral Interface (SPI) circuit is a synchronous serial data link that provides communication with external devices in Master or Slave Mode. It also enables communication between processors if an external processor is connected to the system.

The Serial Peripheral Interface is essentially a shift register that serially transmits data bits to other SPIs. During a data transfer, one SPI system acts as the "master" which controls the data flow, while the other devices act as "slaves" which have data shifted into and out by the master. Different CPUs can take turn being masters (Multiple Master Protocol opposite to Single Master Protocol where one CPU is always the master while all of the others are always slaves) and one master may simultaneously shift data into multiple slaves. However, only one slave may drive its output to write data back to the master at any given time.

A slave device is selected when the master asserts its NSS signal. If multiple slave devices exist, the master generates a separate slave select signal for each slave (NPCS).

The SPI system consists of two data lines and two control lines:

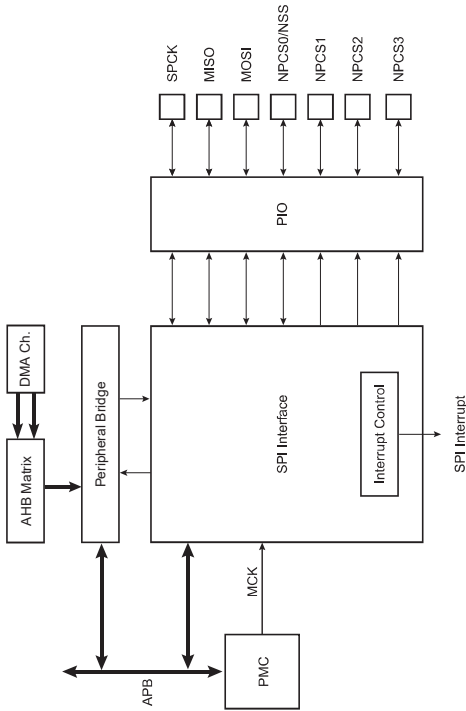
- Master Out Slave In (MOSI): This data line supplies the output data from the master shifted into the input(s) of the slave(s).
- Master In Slave Out (MISO): This data line supplies the output data from a slave to the input of the master. There may be no more than one slave transmitting data during any particular transfer.
- Serial Clock (SPCK): This control line is driven by the master and regulates the flow of the data bits. The master may transmit data at a variety of baud rates; the SPCK line cycles once for each bit that is transmitted.
- Slave Select (NSS): This control line allows slaves to be turned on and off by hardware.

32.2 Embedded Characteristics

- Supports Communication with Serial External Devices
 - Four Chip Selects with External Decoder Support Allow Communication with Up to 15 Peripherals
 - Serial Memories, such as DataFlash and 3-wire EEPROMs
 - Serial Peripherals, such as ADCs, DACs, LCD Controllers, CAN Controllers and Sensors
 - External Co-processors
- Master or Slave Serial Peripheral Bus Interface
 - 8- to 16-bit Programmable Data Length Per Chip Select
 - Programmable Phase and Polarity Per Chip Select
 - Programmable Transfer Delay Between Consecutive Transfers and Delay Before SPI Clock per Chip Select
 - Programmable Delay Between Chip Selects
 - Selectable Mode Fault Detection
- Connection to DMA Channel Capabilities Optimizes Data Transfers
 - One channel for the Receiver, One Channel for the Transmitter

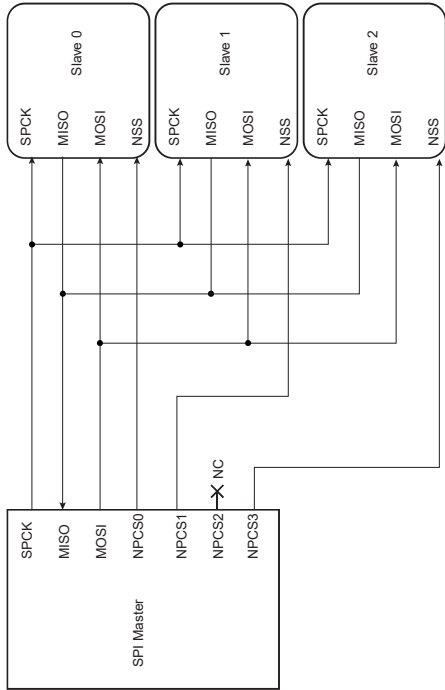
32.3 Block Diagram

Figure 32-1. Block Diagram



32.4 Application Block Diagram

Figure 32-2. Application Block Diagram: Single Master/Multiple Slave Implementation



32.5 Signal Description

Table 32-1. Signal Description			
Pin Name	Pin Description	Type	
MISO	Master In Slave Out	Master Input	Slave Output
MOSI	Master Out Slave In	Output	Input
SCLK	Serial Clock	Output	Input
NPCS1-NPCS3	Peripheral Chip Selects	Output	Unused
NPCS0/NSS	Peripheral Chip Select/Slave Select	Output	Input

32.6 Product Dependencies

32.6.1 I/O Lines

The pins used for interfacing the compliant external devices may be multiplexed with PIO lines. The programmer must first program the PIO controllers to assign the SPI pins to their peripheral functions.

Table 32-2. I/O Lines

Instance	Signal	I/O Line	Peripheral
SPI0	SPI0_MISO	PA25	A
SPI0	SPI0_MOSI	PA26	A
SPI0	SPI0_NPCS0	PA28	A
SPI0	SPI0_NPCS1	PA29	A
SPI0	SPI0_NPCS1	PB20	B
SPI0	SPI0_NPCS2	PA30	A
SPI0	SPI0_NPCS2	PB21	B
SPI0	SPI0_NPCS3	PA31	A
SPI0	SPI0_NPCS3	PB23	B
SPI0	SPI0_SCK	PA27	A
SPI1	SPI1_MISO	PE28	A
SPI1	SPI1_MOSI	PE29	A
SPI1	SPI1_NPCS0	PE31	A
SPI1	SPI1_NPCS1	PF0	A
SPI1	SPI1_NPCS2	PF1	A
SPI1	SPI1_NPCS3	PF2	A
SPI1	SPI1_SCK	PE30	A

32.6.2 Power Management

The SPI may be clocked through the Power Management Controller (PMC), thus the programmer must first configure the PMC to enable the SPI clock.

32.6.3 Interrupt

The SPI interface has an interrupt line connected to the Interrupt Controller. Handling the SPI interrupt requires programming the interrupt controller before configuring the SPI.

Table 32-3. Peripheral IDs

Instance	ID
SPI0	24
SPI1	25

32.7 Functional Description

32.7.1 Modes of Operation

The SPI operates in Master Mode or in Slave Mode.

Operation in Master Mode is programmed by writing at 1 the MSTR bit in the Mode Register. The pins NPCS0 to NPCS3 are all configured as outputs, the SPCK pin is driven, the MISO line is wired on the receiver input and the MOSI line driven as an output by the transmitter.

If the MSTR bit is written at 0, the SPI operates in Slave Mode. The MISO line is driven by the transmitter output, the MOSI line is wired on the receiver input, the SPCK pin is driven by the transmitter to synchronize the receiver. The NPCS0 pin becomes an input, and is used as a Slave Select signal (NSS). The pins NPCS1 to NPCS3 are not driven and can be used for other purposes.

The data transfers are identically programmable for both modes of operations. The baud rate generator is activated only in Master Mode.

32.7.2 Data Transfer

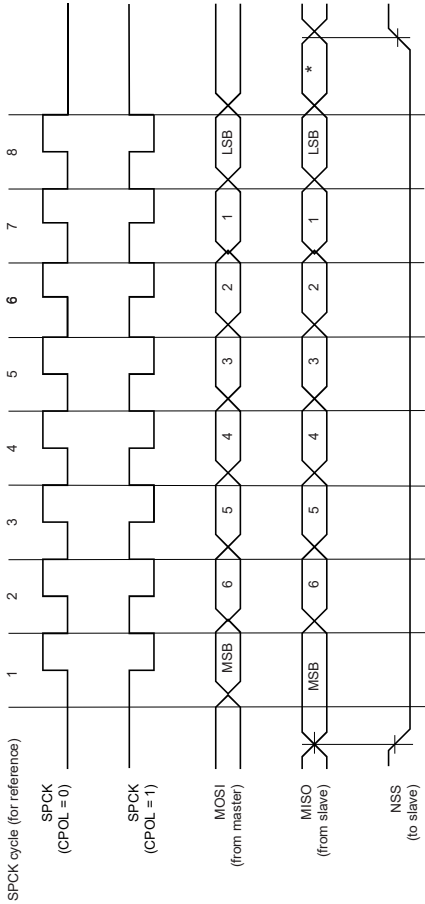
Four combinations of polarity and phase are available for data transfers. The clock polarity is programmed with the CPOL bit in the Chip Select Register. The clock phase is programmed with the NCPHA bit. These two parameters determine the edges of the clock signal on which data is driven and sampled. Each of the two parameters has two possible states, resulting in four possible combinations that are incompatible with one another. Thus, a master/slave pair must use the same parameter pair values to communicate. If multiple slaves are used and fixed in different configurations, the master must reconfigure itself each time it needs to communicate with a different slave.

Table 32-4 shows the four modes and corresponding parameter settings.

Table 32-4. SPI Bus Protocol Mode					
SPI Mode	CPOL	NCPHA	Shift SPCK Edge	Capture SPCK Edge	SPCK Inactive Level
0	0	1	Falling	Rising	Low
1	0	0	Rising	Falling	Low
2	1	1	Rising	Falling	High
3	1	0	Falling	Rising	High

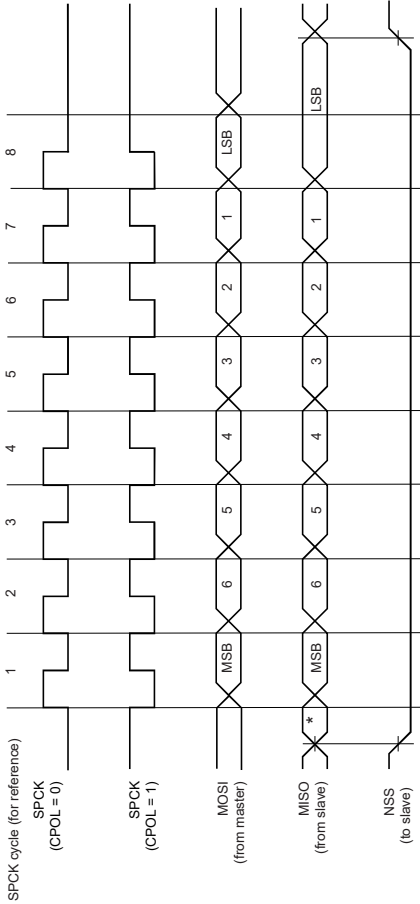
Figure 32-3 and Figure 32-4 show examples of data transfers.

Figure 32-3. SPI Transfer Format (NCPHA = 1, 8 bits per transfer)



* Not defined, but normally MSB of previous character received.

Figure 32-4. SPI Transfer Format (NCPHA = 0, 8 bits per transfer)



* Not defined but normally LSB of previous character transmitted.

32.7.3 Master Mode Operations

When configured in Master Mode, the SPI operates on the clock generated by the internal programmable baud rate generator. It fully controls the data transfers to and from the slave(s) connected to the SPI bus. The SPI drives the chip select line to the slave and the serial clock signal (SPCK).

The SPI features two holding registers, the Transmit Data Register and the Receive Data Register, and a single Shift Register. The holding registers maintain the data flow at a constant rate.

After enabling the SPI, a data transfer begins when the processor writes to the SPI_TDR (Transmit Data Register). The written data is immediately transferred in the Shift Register and transfer on the SPI bus starts. While the data in the Shift Register is shifted on the MOSI line, the MISO line is sampled and shifted in the Shift Register. Receiving data cannot occur without transmitting data. If receiving mode is not needed, for example when communicating with a slave receiver only (such as an LCD), the receive status flags in the status register can be discarded.

Before writing the TDR, the PCS field in the SPI MR register must be set in order to select a slave.

After enabling the SPI, a data transfer begins when the processor writes to the SPI_TDR (Transmit Data Register). The written data is immediately transferred in the Shift Register and transfer on the SPI bus starts. While the data in the Shift Register is shifted on the MOSI line, the MISO line is sampled and shifted in the Shift Register. Transmission cannot occur without reception.

Before writing the TDR, the PCS field must be set in order to select a slave.

If new data is written in SPI_TDR during the transfer, it stays in it until the current transfer is completed. Then, the received data is transferred from the Shift Register to SPI_RDR, the data in SPI_TDR is loaded in the Shift Register and a new transfer starts.

The transfer of a data written in SPI_TDR in the Shift Register is indicated by the TDRE bit (Transmit Data Register Empty) in the Status Register (SPI_SR). When new data is written in SPI_TDR, this bit is cleared. The TDRE bit is used to trigger the Transmit DMA channel.

The end of transfer is indicated by the TXEMPTY flag in the SPI_SR register. If a transfer delay (DLYBCT) is greater than 0 for the last transfer, TXEMPTY is set after the completion of said delay. The master clock (MCK) can be switched off at this time.

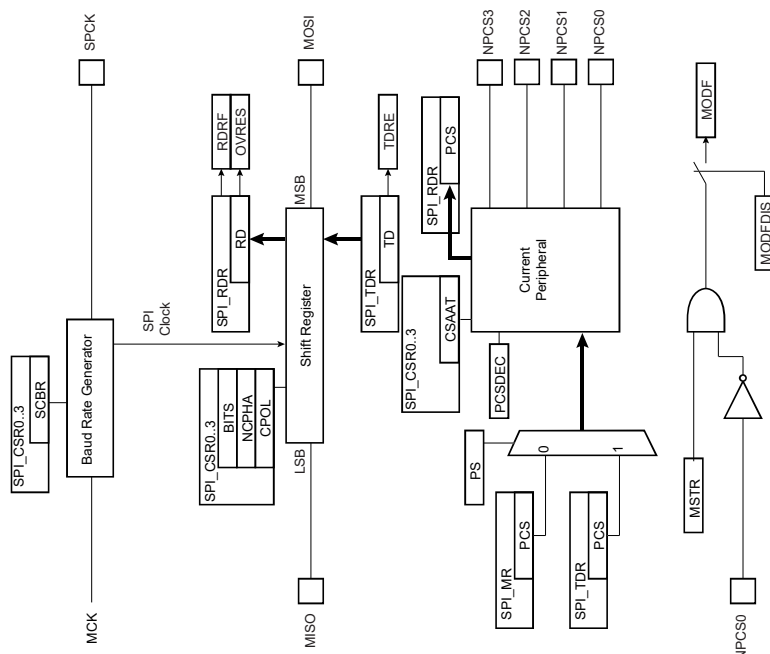
The transfer of received data from the Shift Register in SPI_RDR is indicated by the RDRF bit (Receive Data Register Full) in the Status Register (SPI_SR). When the received data is read, the RDRF bit is cleared.

If the SPI_RDR (Receive Data Register) has not been read before new data is received, the Overrun Error bit (OVRES) in SPI_SR is set. As long as this flag is set, data is loaded in SPI_RDR. The user has to read the status register to clear the OVRES bit.

Figure 32-5, shows a block diagram of the SPI when operating in Master Mode. Figure 32-6 on page 684 shows a flow chart describing how transfers are handled.

32.7.3.1 Master Mode Block Diagram

Figure 32-5. Master Mode Block Diagram



32.7.3.2 Master Mode Flow Diagram

Figure 32-6. Master Mode Flow Diagram

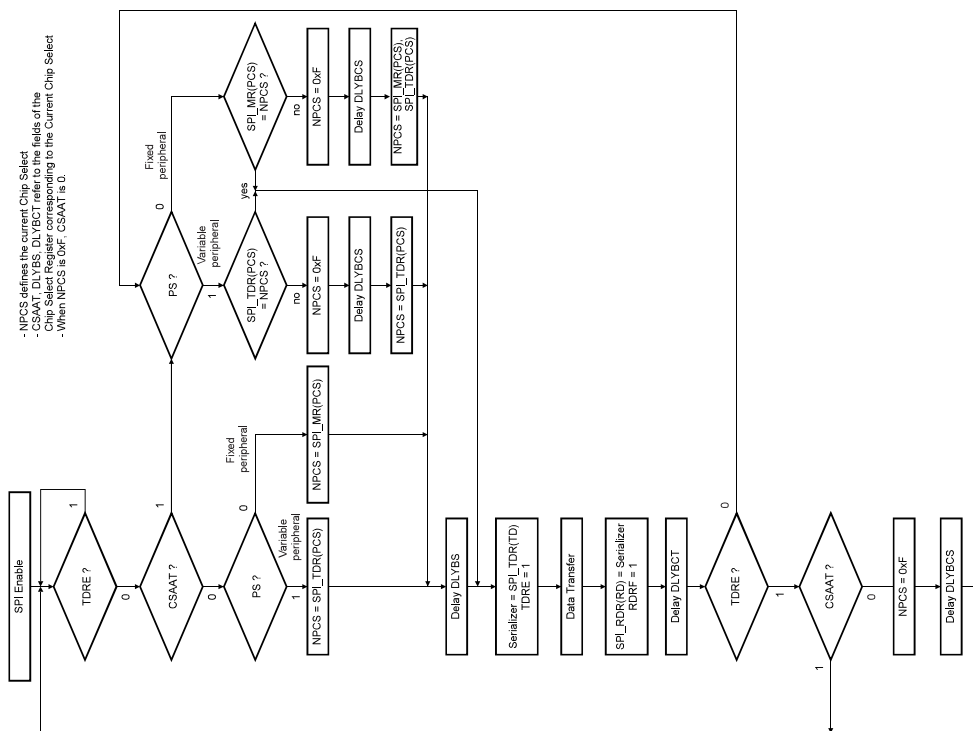
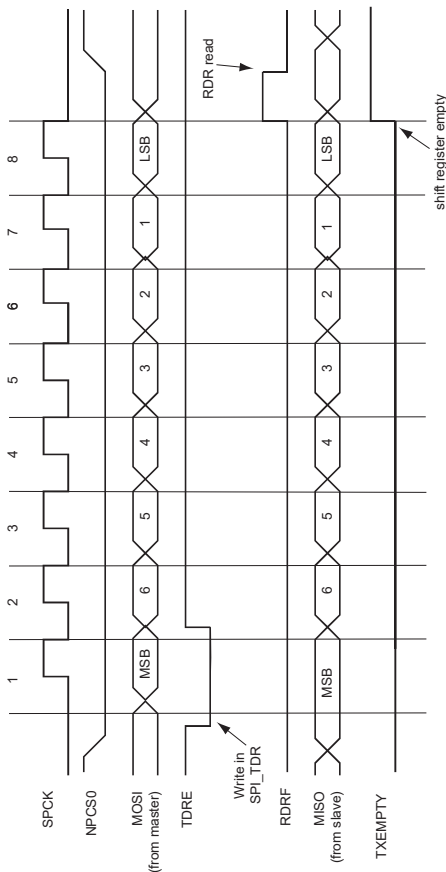


Figure 32-7. Status Register Flags Behavior



32.7.3.3 Clock Generation

The SPI Baud rate clock is generated by dividing the Master Clock (MCK), by a value between 1 and 255. This allows a maximum operating baud rate at up to Master Clock and a minimum operating baud rate of MCK divided by 255.

Programming the SCBR field at 0 is forbidden. Triggering a transfer while SCBR is at 0 can lead to unpredictable results.

At reset, SCBR is 0 and the user has to program it at a valid value before performing the first transfer. The divisor can be defined independently for each chip select, as it has to be programmed in the SCBR field of the Chip Select Registers. This allows the SPI to automatically adapt the baud rate for each interfaced peripheral without reprogramming.

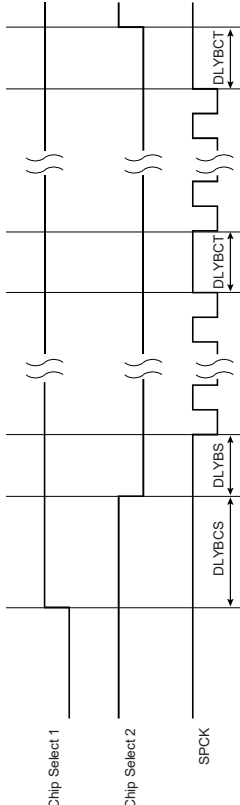
32.7.3.4 Transfer Delays

Figure 32-8 shows a chip select transfer change and consecutive transfers on the same chip select. Three delays can be programmed to modify the transfer waveforms:

- The delay between chip selects, programmable only once for all the chip selects by writing the DLYBCS field in the Mode Register. Allows insertion of a delay between release of one chip select and before assertion of a new one.
- The delay before SPCK, independently programmable for each chip select by writing the field DLYBS. Allows the start of SPCK to be delayed after the chip select has been asserted.
- The delay between consecutive transfers, independently programmable for each chip select by writing the DLYBCT field. Allows insertion of a delay between two transfers occurring on the same chip select

These delays allow the SPI to be adapted to the interfaced peripherals and their speed and bus release time.

Figure 32-8. Programmable Delays



32.7.3.5 Peripheral Selection

The serial peripherals are selected through the assertion of the NPCS0 to NPCS3 signals. By default, all the NPCS signals are high before and after each transfer.

- Fixed Peripheral Select: SPI exchanges data with only one peripheral
- Fixed Peripheral Select is activated by writing the PS bit to zero in SPI_MR (Mode Register). In this case, the current peripheral is defined by the PCS field in SPI_MR and the PCS field in the SPI_TDR has no effect.
- Variable Peripheral Select: Data can be exchanged with more than one peripheral without having to reprogram the NPCS field in the SPI_MR register.

Variable Peripheral Select is activated by setting PS bit to one. The PCS field in SPI_TDR is used to select the current peripheral. This means that the peripheral selection can be defined for each new data. The value to write in the SPI_TDR register as the following format.

[xxxxxx(7-bit) + LASTXFER(1-bit)⁽¹⁾ + PCS (4-bit) + DATA (8 to 16-bit)] with PCS equals to the chip select to assert as defined in [Section 32.8.4](#) (SPI Transmit Data Register) and LASTXFER bit at 0 or 1 depending on CSAAT bit.

Note: 1. Optional.

CSAAT, LASTXFER and CSAAT bits are discussed in [Section 32.7.3.9 "Peripheral Deselection with DMA"](#). If LASTXFER is used, the command must be issued before writing the last character. Instead of LASTXFER, the user can use the SPIDIS command. After the end of the DMA transfer, wait for the TXEMPTY flag, then write SPIDIS into the SPI_CR register (this will not change the configuration register values); the NPCS will be deactivated after the last character transfer. Then, another DMA transfer can be started if the SPIEN was previously written in the SPI_CR register.

32.7.3.6 SPI Direct Access Memory Controller (DMAC)

In both fixed and variable mode the Direct Memory Access Controller (DMAC) can be used to reduce processor overhead.

The Fixed Peripheral Selection allows buffer transfers with a single peripheral. Using the DMAC is an optimal means, as the size of the data transfer between the memory and the SPI is either 8 bits or 16 bits. However, changing the peripheral selection requires the Mode Register to be reprogrammed.

The Variable Peripheral Selection allows buffer transfers with multiple peripherals without reprogramming the Mode Register. Data written in SPI_TDR is 32 bits wide and defines the real data to be transmitted and the peripheral it is destined to. Using the DMAC in this mode requires 32-bit wide buffers, with the data in the LSBs and the PCS and LASTXFER fields in the MSBs, however the SPI still controls the number of bits (8 to16) to be

transferred through MISO and MOSI lines with the chip select configuration registers. This is not the optimal means in term of memory size for the buffers, but it provides a very effective means to exchange data with several peripherals without any intervention of the processor.

32.7.3.7 Peripheral Chip Select Decoding

The user can program the SPI to operate with up to 15 peripherals by decoding the four Chip Select lines, NPCS0 to NPCS3 with 1 of up to 16 decoder/demultiplexer. This can be enabled by writing the PCSDEC bit at 1 in the Mode Register (SPI_MR).

When operating without decoding, the SPI makes sure that in any case only one chip select line is activated, i.e., one NPCS line driven low at a time. If two bits are defined low in a PCS field, only the lowest numbered chip select is driven low.

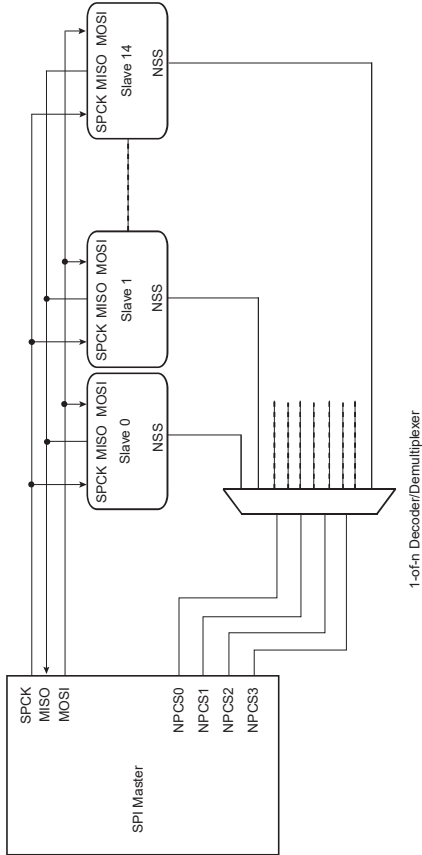
When operating with decoding, the SPI directly outputs the value defined by the PCS field on NPCS lines of either the Mode Register or the Transmit Data Register (depending on PS).

As the SPI sets a default value of 0xF on the chip select lines (i.e. all chip select lines at 1) when not processing any transfer, only 15 peripherals can be decoded.

The SPI has only four Chip Select Registers, not 15. As a result, when decoding is activated, each chip select defines the characteristics of up to four peripherals. As an example, SPI_CRSD defines the characteristics of the externally decoded peripherals 0 to 3, corresponding to the PCS values 0x0 to 0x3. Thus, the user has to make sure to connect compatible peripherals on the decoded chip select lines 0 to 3, 4 to 7, 8 to 11 and 12 to 14. [Figure 32-9](#) below shows such an implementation.

If the CSAAT bit is used, with or without the DMAC, the Mode Fault detection for NPCS0 line must be disabled. This is not needed for all other chip select lines since Mode Fault Detection is only on NPCS0.

Figure 32-9. Chip Select Decoding Application Block Diagram: Single Master/Multiple Slave Implementation



32.7.3.8 Peripheral Deselection without DMAC

During a transfer of more than one data on a Chip Select without the DMAC, the SPI_TDR is loaded by the processor, the flag TDRE rises as soon as the content of the SPI_TDR is transferred into the internal shift register. When this flag is detected high, the SPI_TDR can be reloaded. If this reload by the processor occurs before the end of the current transfer and if the next transfer is performed on the same chip select as the current transfer, the Chip Select is not de-asserted between the two transfers. But depending on the application software handling the SPI status register flags (by interrupt or polling method) or servicing other interrupts or other tasks, the processor may not reload the SPI_TDR in time to keep the chip select active (low). A null Delay Between Consecutive Transfer (DLYBCT) value in the SPI_CSR register, will give even less time for the processor to reload the SPI_TDR. With some SPI slave peripherals, requiring the chip select line to remain active (low) during a full set of transfers might lead to communication errors.

To facilitate interfacing with such devices, the Chip Select Register [CSR0...CSR3] can be programmed with the CSAAT bit (Chip Select Active After Transfer) at 1. This allows the chip select lines to remain in their current state (low = active) until transfer to another chip select is required. Even if the SPI_TDR is not reloaded the chip select will remain active. To have the chip select line to raise at the end of the transfer the Last transfer Bit (LASTXFER) in the SPI_MR register must be set at 1 before writing the last data to transmit into the SPI_TDR.

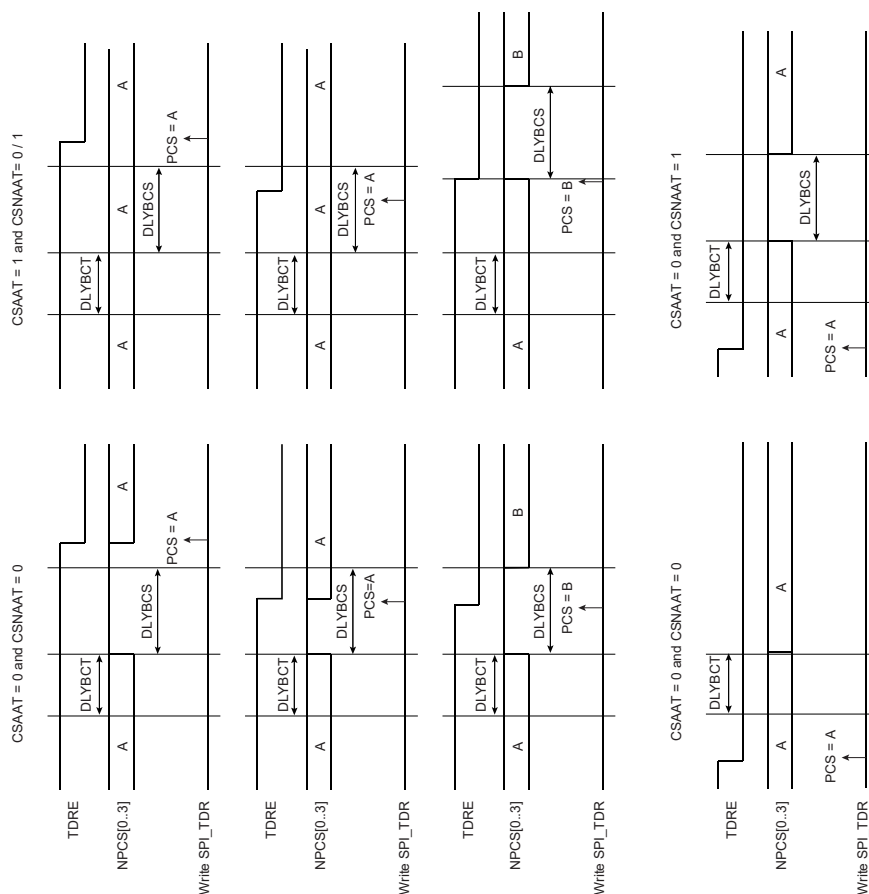
32.7.3.9 Peripheral Deselection with DMAC

When the Direct Memory Access Controller is used, the chip select line will remain low during the whole transfer since the TDRE flag is managed by the DMAC itself. The reloading of the SPI_TDR by the DMAC is done as soon as TDRE flag is set to one. In this case the use of CSAAT bit might not be needed. However, it may happen that when other DMAC channels connected to other peripherals are in use as well, the SPI DMAC might be delayed by another (DMAC with a higher priority on the bus). Having DMAC buffers in slower memories like flash memory or SDRAM compared to fast internal SRAM, may lengthen the reload time of the SPI_TDR by the DMAC as well. This means that the SPI_TDR might not be reloaded in time to keep the chip select line low. In this case the chip select line may toggle between data transfer and according to some SPI Slave devices, the communication might get lost. The use of the CSAAT bit might be needed.

When the CSAAT bit is set at 0, the NPCS does not rise in all cases between two transfers on the same peripheral. During a transfer on a Chip Select, the flag TDRE rises as soon as the content of the SPI_TDR is transferred into the internal shifter. When this flag is detected the SPI_TDR can be reloaded. If this reload occurs before the end of the current transfer and if the next transfer is performed on the same chip select as the current transfer, the Chip Select is not de-asserted between the two transfers. This might lead to difficulties for interfacing with some serial peripherals requiring the chip select to be de-asserted after each transfer. To facilitate interfacing with such devices, the Chip Select Register can be programmed with the CSAAT bit (Chip Select Not Active After Transfer) at 1. This allows to de-assert systematically the chip select lines during a time DLYBCT. (The value of the CSAAT bit is taken into account only if the CSAAT bit is set at 0 for the same Chip Select).

Figure 32-10 shows different peripheral deselection cases and the effect of the CSAAT and CSAAT bits.

Figure 32-10. Peripheral Deselection



32.7.3.10 Mode Fault Detection

A mode fault is detected when the SPI is programmed in Master Mode and a low level is driven by an external master on the NPCS0/NSS signal. In this case, multi-master configuration, NPCS0, MOSI, MISO and SPCK pins must be configured in open drain (through the PIO controller). When a mode fault is detected, the MODF bit in the SPI_SR is set until the SPI_SR is read and the SPI is automatically disabled until re-enabled by writing the SPIEN bit in the SPI_CR (Control Register) at 1.

32.7.5 Write Protected Registers

To prevent any single software error that may corrupt SPI behavior, the registers listed below can be write-protected by setting the WPEN bit in the SPI Write Protection Mode Register (SPI_WPMR).

If a write access in a write-protected register is detected, then the WPVS flag in the SPI Write Protection Status Register (SPI_WPSR) is set and the field WPVSR indicates in which register the write access has been attempted.

The WPVS flag is automatically reset after reading the SPI Write Protection Status Register (SPI_WPSR).

List of the write-protected registers:

Section 32.8.2 "SPI Mode Register"

Section 32.8.9 "SPI Chip Select Register"

By default, the Mode Fault detection circuitry is enabled. The user can disable Mode Fault detection by setting the MODFDIS bit in the SPI Mode Register (SPI_MR).

32.7.4 SPI Slave Mode

When operating in Slave Mode, the SPI processes data bits on the clock provided on the SPI clock pin (SPCK).

The SPI waits for NSS to go active before receiving the serial clock from an external master. When NSS falls, the serial clock is validated on the serializer, which processes the number of bits defined by the BITS field of the Chip Select Register 0 (SPI_CS0R0). These bits are processed following a phase and a polarity defined respectively by the NCPHA and CPOL bits of the SPI_CS0R0. Note that BITS, CPOL and NCPHA of the other Chip Select Registers have no effect when the SPI is programmed in Slave Mode.

The bits are shifted out on the MISO line and sampled on the MOSI line.

(For more information on BITS field, see also, the [\(Notes\)](#) below the register table; [Section 32.8.9 "SPI Chip Select Register"](#) on page 703.)

When all the bits are processed, the received data is transferred in the Receive Data Register and the RDRF bit is set. When the RDRF bit is set, the user has to read the data from the Receive Data Register. If the SPI_RDR (Receive Data Register) has not been read before new data is received, the Overrun Error flag (OVRER) is set. As long as this flag is set, data is loaded in SPI_RDR. The user has to read the data from the SPI_RDR register to clear the OVRER bit.

When a transfer starts, the data shifted out is the data present in the Shift Register. If no data has been written in the Transmit Data Register (SPI_TDR), the last data received is transferred. If no data has been received since the last reset, all bits are transmitted low, as the Shift Register resets at 0.

If a new data is written in SPI_TDR, it is transferred immediately in the Shift Register and the TDRE bit rises. If a new data is written in SPI_TDR until a transfer occurs, i.e. NSS falls and there is a valid clock on the SSPECK pin. When the transfer occurs, the last data written in SPI_TDR is transferred in the Shift Register and the TDRE bit rises. This enables frequent updates of critical variables with single transfers.

Then, a new data is loaded in the Shift Register from the Transmit Data Register. In case no character is ready to be transmitted, i.e. no character has been written in SPI_TDR since the last load from SPI_TDR to the Shift Register, the Shift Register is not modified and the last received character is retransmitted. In this case the Underrun Error Status Flag (UNDES) is set in the SPI_SR.

Figure 32-11 shows a block diagram of the SPI when operating in Slave Mode.

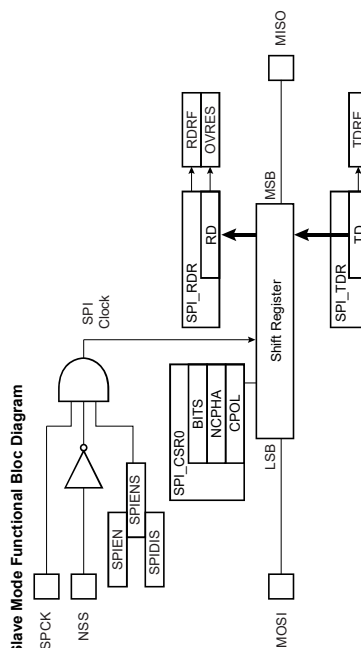


Figure 32-11. Slave Mode Functional Bloc Diagram

32.8 Serial Peripheral Interface (SPI) User Interface

Table 32-5. Register Mapping

Offset	Register	Name	Access	Reset
0x00	Control Register	SPI_CR	Write-only	---
0x04	Mode Register	SPI_MR	Read-write	0x0
0x08	Receive Data Register	SPI_RDR	Read-only	0x0
0x0C	Transmit Data Register	SPI_TDR	Write-only	---
0x10	Status Register	SPI_SR	Read-only	0x000000F0
0x14	Interrupt Enable Register	SPI_IER	Write-only	---
0x18	Interrupt Disable Register	SPI_IDR	Write-only	---
0x1C	Interrupt Mask Register	SPI_IMR	Read-only	0x0
0x20 - 0x2C	Reserved			
0x30	Chip Select Register 0	SPI_CSR0	Read-write	0x0
0x34	Chip Select Register 1	SPI_CSR1	Read-write	0x0
0x38	Chip Select Register 2	SPI_CSR2	Read-write	0x0
0x3C	Chip Select Register 3	SPI_CSR3	Read-write	0x0
0x4C - 0xE0	Reserved	-	-	-
0xE4	Write Protection Control Register	SPI_WPMR	Read-write	0x0
0xE8	Write Protection Status Register	SPI_WPSR	Read-only	0x0
0x00E8 - 0x00F8	Reserved	-	-	-
0x00FC	Reserved	-	-	-

32.8.1 SPI Control Register

Name: SPI_CR

Address: 0x40008000 (0), 0x4000C000 (1)

Access: Write-only

31	30	29	28	27	26	25	24
---	---	---	---	---	---	---	LASTXFER
23	22	21	20	19	18	17	16
---	---	---	---	---	---	---	---
15	14	13	12	11	10	9	8
---	---	---	---	---	---	---	---
7	6	5	4	3	2	1	0
SWRST	---	---	---	---	---	SPIDIS	SPHEN

• SPIEN: SPI Enable

0 = No effect.
1 = Enables the SPI to transfer and receive data.

• SPIDIS: SPI Disable

0 = No effect.
1 = Disables the SPI.
As soon as SPIDIS is set, SPI finishes its transfer.
All pins are set in input mode and no data is received or transmitted.

If a transfer is in progress, the transfer is finished before the SPI is disabled.
If both SPHEN and SPIDIS are equal to one when the control register is written, the SPI is disabled.

• SWRST: SPI Software Reset

0 = No effect.
1 = Reset the SPI. A software-triggered hardware reset of the SPI interface is performed.
The SPI is in slave mode after software reset.
DMAC channels are not affected by software reset.

• LASTXFER: Last Transfer

0 = No effect.
1 = The current NPCS will be deasserted after the character written in TD has been transferred. When CSAAT is set, this allows to close the communication with the current serial peripheral by raising the corresponding NPCS line as soon as TD transfer has completed.
Refer to [Section 32.7.3.5 "Peripheral Selection"](#) for more details.

32.8.2 SPI Mode Register

Name: SPI_MR
Address: 0x40000004 (0), 0x4000C004 (1)
Access: Read-write

31	30	29	28	27	26	25	24
DLYBCS							
23	22	21	20	19	18	17	16
—	—	—	—	—	PCS	—	—
15	14	13	12	11	10	9	8
—	—	—	—	—	—	—	—
7	6	5	4	3	2	1	0
LLB	—	WDRBT	MODFDIS	—	PCSDEC	PS	MSTR

This register can only be written if the WPEN bit is cleared in "SPI Write Protection Mode Register".

- **MSTR: Master/Slave Mode**
0 = SPI is in Slave mode.
1 = SPI is in Master mode.

- **PS: Peripheral Select**
0 = Fixed Peripheral Select.
1 = Variable Peripheral Select.

- **PCSDEC: Chip Select Decode**
0 = The chip selects are directly connected to a peripheral device.
1 = The four chip select lines are connected to a 4- to 16-bit decoder.

When PCSDEC equals one, up to 15 Chip Select signals can be generated with the four lines using an external 4- to 16-bit decoder. The Chip Select Registers define the characteristics of the 15 chip selects according to the following rules:

- SPI_CSR0 defines peripheral chip select signals 0 to 3.
- SPI_CSR1 defines peripheral chip select signals 4 to 7.
- SPI_CSR2 defines peripheral chip select signals 8 to 11.
- SPI_CSR3 defines peripheral chip select signals 12 to 14.

- **MODFDIS: Mode Fault Detection**
0 = Mode fault detection is enabled.
1 = Mode fault detection is disabled.

- **WDRBT: Wait Data Read Before Transfer**
0 = No Effect. In master mode, a transfer can be initiated whatever the state of the Receive Data Register is.
1 = In Master Mode, a transfer can start only if the Receive Data Register is empty, i.e. does not contain any unread data. This mode prevents overrun error in reception.

- **LLB: Local Loopback Enable**
0 = Local loopback path disabled.
1 = Local loopback path enabled
LLB controls the local loopback on the data serializer for testing in Master Mode only. (MISO is internally connected on MOSI.)

- **PCS: Peripheral Chip Select**
This field is only used if Fixed Peripheral Select is active (PS = 0).
If PCSDEC = 0:

- PCS = xxx0 NPCS[3:0] = 1110
- PCS = xx01 NPCS[3:0] = 1101
- PCS = x011 NPCS[3:0] = 1011
- PCS = 0111 NPCS[3:0] = 0111
- PCS = 1111 forbidden (no peripheral is selected)
- (x = don't care)

If PCSDEC = 1:
NPCS[3:0] output signals = PCS.

- **DLYBCS: Delay Between Chip Selects**
This field defines the delay from NPCS inactive to the activation of another NPCS. The DLYBCS time guarantees non-overlapping chip selects and solves bus contentions in case of peripherals having long data float times.
If DLYBCS is less than or equal to six, six MCK periods will be inserted by default.
Otherwise, the following equation determines the delay:

Delay Between Chip Selects = $\frac{DLYBCS}{MCK}$

32.8.3 SPI Receive Data Register

Name: SPI_RDR
Address: 0x40008008 (0), 0x4000C008 (1)
Access: Read-only

31	30	29	28	27	26	25	24
—	—	—	—	—	—	—	—
23	22	21	20	19	18	17	16
—	—	—	—	—	PCS	—	—
15	14	13	12	11	10	9	8
RD							
7	6	5	4	3	2	1	0
RD							

- **RD: Receive Data**
Data received by the SPI Interface is stored in this register right-justified. Unused bits read zero.

• **PCS: Peripheral Chip Select**

In Master Mode only, these bits indicate the value on the NPCS pins at the end of a transfer. Otherwise, these bits read zero.
Note: When using variable peripheral select mode (PS = 1 in SPI_MR) it is mandatory to also set the WDRBT field to 1 if the SPI_RDR PCS field is to be processed.

32.8.4 SPI Transmit Data Register

Name: SPI_TDR
Address: 0x4000800C (0), 0x4000C00C (1)
Access: Write-only

31	30	29	28	27	26	25	24
—	—	—	—	—	—	—	LASTXFER
23	22	21	20	19	18	17	16
—	—	—	—	—	PCS	—	—
15	14	13	12	11	10	9	8
TD							
7	6	5	4	3	2	1	0
TD							

- **TD: Transmit Data**
Data to be transmitted by the SPI Interface is stored in this register. Information to be transmitted must be written to the transmit data register in a right-justified format.

• **PCS: Peripheral Chip Select**

This field is only used if Variable Peripheral Select is active (PS = 1).
If PCSDEC = 0:

- PCS = xxx0 NPCS[3:0] = 1110
- PCS = xx01 NPCS[3:0] = 1101
- PCS = x011 NPCS[3:0] = 1011
- PCS = 0111 NPCS[3:0] = 0111
- PCS = 1111 forbidden (no peripheral is selected)
(x = don't care)

If PCSDEC = 1:

NPCS[3:0] output signals = PCS

• **LASTXFER: Last Transfer**

0 = No effect.
1 = The current NPCS will be deasserted after the character written in TD has been transferred. When CSAAT is set, this allows to close the communication with the current serial peripheral by raising the corresponding NPCS line as soon as TD transfer has completed.
This field is only used if Variable Peripheral Select is active (PS = 1).

• **SPIENS: SPI Enable Status**

0 = SPI is disabled.
1 = SPI is enabled.

32.8.5 SPI Status Register

Name: SPI_SR
Address: 0x40008010 (0), 0x4000C010 (1)
Access: Read-only

31	30	29	28	27	26	25	24
—	—	—	—	—	—	—	—
23	22	21	20	19	18	17	16
—	—	—	—	—	—	—	SPIENS
15	14	13	12	11	10	9	8
—	—	—	—	—	UNDES	TXEMPTY	NSSR
7	6	5	4	3	2	1	0
—	—	—	—	OVRES	MODF	TDRE	RDRF

• **RDRF: Receive Data Register Full**

0 = No data has been received since the last read of SPI_RDR
1 = Data has been received and the received data has been transferred from the serializer to SPI_RDR since the last read of SPI_RDR.

• **TDRE: Transmit Data Register Empty**

0 = Data has been written to SPI_TDR and not yet transferred to the serializer.
1 = The last data written in the Transmit Data Register has been transferred to the serializer.
TDRE equals zero when the SPI is disabled or at reset. The SPI enable command sets this bit to one.

• **MODF: Mode Fault Error**

0 = No Mode Fault has been detected since the last read of SPI_SR.
1 = A Mode Fault occurred since the last read of the SPI_SR.

• **OVRES: Overrun Error Status**

0 = No overrun has been detected since the last read of SPI_SR.
1 = An overrun has occurred since the last read of SPI_SR.
An overrun occurs when SPI_RDR is loaded at least twice from the serializer since the last read of the SPI_RDR.

• **NSSR: NSS Rising**

0 = No rising edge detected on NSS pin since last read.
1 = A rising edge occurred on NSS pin since last read.

• **TXEMPTY: Transmission Registers Empty**

0 = As soon as data is written in SPI_TDR.
1 = SPI_TDR and internal shifter are empty. If a transfer delay has been defined, TXEMPTY is set after the completion of such delay.

• **UNDES: Underrun Error Status (Slave Mode Only)**

0 = No underrun has been detected since the last read of SPI_SR.
1 = A transfer begins whereas no data has been loaded in the Transmit Data Register.

32.8.6 SPI Interrupt Enable Register

Name: SPI_IER
Address: 0x40008014 (0), 0x4000C014 (1)
Access: Write-only

31	30	29	28	27	26	25	24
—	—	—	—	—	—	—	—
23	22	21	20	19	18	17	16
—	—	—	—	—	—	—	—
15	14	13	12	11	10	9	8
—	—	—	—	—	UNDES	TXEMPTY	NSSR
7	6	5	4	3	2	1	0
—	—	—	—	OVRES	MODF	TDRE	RDRF

0 = No effect.
1 = Enables the corresponding interrupt.

- RDRF: Receive Data Register Full Interrupt Enable
- TDRE: SPI Transmit Data Register Empty Interrupt Enable
- MODF: Mode Fault Error Interrupt Enable
- OVRES: Overrun Error Interrupt Enable
- NSSR: NSS Rising Interrupt Enable
- TXEMPTY: Transmission Registers Empty Enable
- UNDES: Underrun Error Interrupt Enable

32.8.7 SPI Interrupt Disable Register

Name: SPI_IDR
Address: 0x40008018 (0), 0x4000C018 (1)
Access: Write-only

31	30	29	28	27	26	25	24
—	—	—	—	—	—	—	—
23	22	21	20	19	18	17	16
—	—	—	—	—	—	—	—
15	14	13	12	11	10	9	8
—	—	—	—	—	UNDES	TXEMPTY	NSSR
7	6	5	4	3	2	1	0
—	—	—	—	OVRES	MODF	TDRE	RDRF

0 = No effect.
1 = Disables the corresponding interrupt.

- RDRF: Receive Data Register Full Interrupt Disable
- TDRE: SPI Transmit Data Register Empty Interrupt Disable
- MODF: Mode Fault Error Interrupt Disable
- OVRES: Overrun Error Interrupt Disable
- NSSR: NSS Rising Interrupt Disable
- TXEMPTY: Transmission Registers Empty Disable
- UNDES: Underrun Error Interrupt Disable

32.8.8 SPI Interrupt Mask Register

Name: SPI_IMR
Address: 0x4000801C (0), 0x4000C01C (1)
Access: Read-only

31	30	29	28	27	26	25	24
—	—	—	—	—	—	—	—
23	22	21	20	19	18	17	16
—	—	—	—	—	—	—	—
15	14	13	12	11	10	9	8
—	—	—	—	—	UNDES	TXEMPTY	NSSR
7	6	5	4	3	2	1	0
—	—	—	—	OVRES	MODF	TDRE	RDRF

0 = The corresponding interrupt is not enabled.
1 = The corresponding interrupt is enabled.

- RDRF: Receive Data Register Full Interrupt Mask
- TDRE: SPI Transmit Data Register Empty Interrupt Mask
- MODF: Mode Fault Error Interrupt Mask
- OVRES: Overrun Error Interrupt Mask
- NSSR: NSS Rising Interrupt Mask
- TXEMPTY: Transmission Registers Empty Mask
- UNDES: Underrun Error Interrupt Mask



32.8.9 SPI Chip Select Register

Name: SPI_CSR[x=0..3]
Address: 0x40008030 (0), 0x4000C030 (1)
Access: Read/Write

31	30	29	28	27	26	25	24
DLYBCT							
23	22	21	20	19	18	17	16
DLYBS							
15	14	13	12	11	10	9	8
SCBR							
7	6	5	4	3	2	1	0
BITS		CSAAT		CSNAAT		NCPHA	
						CPOL	

This register can only be written if the WPEN bit is cleared in "SPI Write Protection Mode Register".
Note: SPI_CSRx registers must be written even if the user wants to use the defaults. The BITS field will not be updated with the translated value unless the register is written.

- CPOL: Clock Polarity
0 = The inactive state value of SPCK is logic level zero.
1 = The inactive state value of SPCK is logic level one.
CPOL is used to determine the inactive state value of the serial clock (SPCK). It is used with NCPHA to produce the required clock/data relationship between master and slave devices.
- NCPHA: Clock Phase
0 = Data is changed on the leading edge of SPCK and captured on the following edge of SPCK.
1 = Data is captured on the leading edge of SPCK and changed on the following edge of SPCK.
NCPHA determines which edge of SPCK causes data to change and which edge causes data to be captured. NCPHA is used with CPOL to produce the required clock/data relationship between master and slave devices.
- CSAAT: Chip Select Not Active After Transfer (ignored if CSAAT = 1)
0 = The Peripheral Chip Select does not rise between two transfers if the SPI_TDR is reloaded before the end of the first transfer and if the two transfers occur on the same Chip Select.
1 = The Peripheral Chip Select rises systematically after each transfer performed on the same slave. It remains active after the end of transfer for a minimal duration of:
$$-\frac{DLYBCT}{MCK}$$
 (if DLYBCT field is different from 0)
$$-\frac{DLYBCT+1}{MCK}$$
 (if DLYBCT field equals 0)
- CSAAT: Chip Select Active After Transfer
0 = The Peripheral Chip Select Line rises as soon as the last transfer is achieved.
1 = The Peripheral Chip Select does not rise after the last transfer is achieved. It remains active until a new transfer is requested on a different chip select.



• **BITS: Bits Per Transfer**

(See the [Note](#) below the register table; [Section 32.8.9 “SPI Chip Select Register”](#) on page 703.)

The BITS field determines the number of data bits transferred. Reserved values should not be used.

Value	Name	Description
0	8_BIT	8 bits for transfer
1	9_BIT	9 bits for transfer
2	10_BIT	10 bits for transfer
3	11_BIT	11 bits for transfer
4	12_BIT	12 bits for transfer
5	13_BIT	13 bits for transfer
6	14_BIT	14 bits for transfer
7	15_BIT	15 bits for transfer
8	16_BIT	16 bits for transfer
9	—	Reserved
10	—	Reserved
11	—	Reserved
12	—	Reserved
13	—	Reserved
14	—	Reserved
15	—	Reserved

• **SCBR: Serial Clock Baud Rate**

In Master Mode, the SPI Interface uses a modulus counter to derive the SPCK baud rate from the Master Clock MCK. The Baud rate is selected by writing a value from 1 to 255 in the SCBR field. The following equations determine the SPCK baud rate:

$$\text{SPCK Baudrate} = \frac{MCK}{SCBR}$$

Programming the SCBR field at 0 is forbidden. Triggering a transfer while SCBR is at 0 can lead to unpredictable results. At reset, SCBR is 0 and the user has to program it at a valid value before performing the first transfer.

Note: If one of the SCBR fields in SPI_CSRx is set to 1, the other SCBR fields in SPI_CSRx must be set to 1 as well, if they are required to process transfers. If they are not used to transfer data, they can be set at any value.

• **DLYBS: Delay Before SPCK**

This field defines the delay from NPCS valid to the first valid SPCK transition.

When DLYBS equals zero, the NPCS valid to SPCK transition is 1/2 the SPCK clock period.

Otherwise, the following equations determine the delay:

$$\text{Delay Before SPCK} = \frac{DLYBS}{MCK}$$

• **DLYBCT: Delay Between Consecutive Transfers**

This field defines the delay between two consecutive transfers with the same peripheral without removing the chip select. The delay is always inserted after each transfer and before removing the chip select if needed.

When DLYBCT equals zero, no delay between consecutive transfers is inserted and the clock keeps its duty cycle over the character transfers.

Otherwise, the following equation determines the delay:

$$\text{Delay Between Consecutive Transfers} = \frac{32 \times DLYBCT}{MCK}$$

32.8.10 SPI Write Protection Mode Register

Name: SPI_WPMR
Address: 0x400080E4 (0), 0x4000C0E4 (1)
Access: Read-write

31	30	29	28	27	26	25	24
			WPKEY				
23	22	21	20	19	18	17	16
			WPKEY				
15	14	13	12	11	10	9	8
			WPKEY				
7	6	5	4	3	2	1	0
-	-	-	-	-	-	-	WPEN

- WPEN: Write Protection Enable
0: The Write Protection is Disabled
1: The Write Protection is Enabled

WPKEY: Write Protection Key Password

If a value is written in WPEN, the value is taken into account only if WPKEY is written with "SPI" (SPI written in ASCII Code, ie 0x535049 in hexadecimal).

List of the write-protected registers:

- [Section 32.8.2 "SPI Mode Register"](#)
- [Section 32.8.9 "SPI Chip Select Register"](#)

32.8.11 SPI Write Protection Status Register

Name: SPI_WPSR
Address: 0x400080E8 (0), 0x4000C0E8 (1)
Access: Read-only

31	30	29	28	27	26	25	24
-	-	-	-	-	-	-	-
23	22	21	20	19	18	17	16
-	-	-	-	-	-	-	-
15	14	13	12	11	10	9	8
			WPVSR				
7	6	5	4	3	2	1	0
-	-	-	-	-	-	-	WPVS

WPVS: Write Protection Violation Status

0 = No Write Protect Violation has occurred since the last read of the SPI_WPSR register.
1 = A Write Protect Violation has occurred since the last read of the SPI_WPSR register. If this violation is an unauthorized attempt to write a protected register, the associated violation is reported into field WPVSR.

WPVSR: Write Protection Violation Source

This Field indicates the APB Offset of the register concerned by the violation (SPI_MR or SPI_CSR).

ARM® and Thumb®-2 Instruction Set Quick Reference Card

Key to Tables			
Rm {, <opsh>}	See Table Register, optionally shifted by constant	<reglist>	A comma-separated list of registers, enclosed in braces { and }.
<Operand2>	See Table Flexible Operand 2 . Shift and rotate are only available as part of Operand2.	<reglist-PC>	As <reglist>, must not include the PC.
<fields>	See Table PSR fields .	<reglist+PC>	As <reglist>, including the PC.
<PSR>	Either CPSR (Current Processor Status Register) or SPSR (Saved Processor Status Register)	+/-	+ or -. (+ may be omitted.)
C*, V*	Flag is unpredictable in Architecture v4 and earlier, unchanged in Architecture v5 and later.	\$	See Table ARM architecture versions .
<Rs sh>	Can be Rs or an immediate shift value. The values allowed for each shift type are the same as those shown in Table Register, optionally shifted by constant .	<iflags>	Interrupt flags. One or more of a, i, f (abort, interrupt, fast interrupt).
x, y	B meaning half-register [15:0], or T meaning [31:16].	<p_mode>	See Table Processor Modes
<imm8m>	ARM: a 32-bit constant, formed by right-rotating an 8-bit value by an even number of bits. Thumb: a 32-bit constant, formed by left-shifting an 8-bit value by any number of bits, or a bit pattern of one of the forms 0xXYXYXYXY, 0x00XY00XY or 0xXY00XY00.	SPm	SP for the processor mode specified by <p_mode>
<prefix>	See Table Prefixes for Parallel instructions	<lsb>	Least significant bit of bitfield.
{IA IB DA DB}	Increment After, Increment Before, Decrement After, or Decrement Before. IB and DA are not available in Thumb state. If omitted, defaults to IA.	<width>	Width of bitfield. <width> + <lsb> must be <= 32.
<size>	B, SB, H, or SH, meaning Byte, Signed Byte, Halfword, and Signed Halfword respectively. SB and SH are not available in STR instructions.	{X}	RsX is Rs rotated 16 bits if X present. Otherwise, RsX is Rs.
		{!}	Updates base register after data transfer if ! present (pre-indexed).
		{S}	Updates condition flags if S present.
		{T}	User mode privilege if T present.
		{R}	Rounds result to nearest if R present, otherwise truncates result.

Operation		\$	Assembler	S updates	Action	Notes
Add	Add		ADD{S} Rd, Rn, <Operand2>	N Z C V	Rd := Rn + Operand2	N
	with carry		ADC{S} Rd, Rn, <Operand2>	N Z C V	Rd := Rn + Operand2 + Carry	N
	wide	T2	ADD Rd, Rn, #<imm12>		Rd := Rn + imm12, imm12 range 0-4095	T, P
	saturating {doubled}	SE	Q{D}ADD Rd, Rm, Rn		Rd := SAT(Rm + Rn) doubled: Rd := SAT(Rm + SAT(Rn * 2))	Q
Address	Form PC-relative address		ADR Rd, <label>		Rd := <label>, for <label> range from current instruction see Note L	N, L
Subtract	Subtract		SUB{S} Rd, Rn, <Operand2>	N Z C V	Rd := Rn - Operand2	N
	with carry		SBC{S} Rd, Rn, <Operand2>	N Z C V	Rd := Rn - Operand2 - NOT(Carry)	N
	wide	T2	SUB Rd, Rn, #<imm12>	N Z C V	Rd := Rn - imm12, imm12 range 0-4095	T, P
	reverse subtract		RSB{S} Rd, Rn, <Operand2>	N Z C V	Rd := Operand2 - Rn	N
	reverse subtract with carry		RSC{S} Rd, Rn, <Operand2>	N Z C V	Rd := Operand2 - Rn - NOT(Carry)	A
	saturating {doubled}	SE	Q{D}SUB Rd, Rm, Rn		Rd := SAT(Rm - Rn) doubled: Rd := SAT(Rm - SAT(Rn * 2))	Q
	Exception return without stack		SUBS PC, LR, #<imm8>		PC = LR - imm8, CPSR = SPSR(current mode), imm8 range 0-255.	T
Parallel arithmetic	Halfword-wise addition	6	<prefix>ADD16 Rd, Rn, Rm		Rd[31:16] := Rn[31:16] + Rm[31:16], Rd[15:0] := Rn[15:0] + Rm[15:0]	G
	Halfword-wise subtraction	6	<prefix>SUB16 Rd, Rn, Rm		Rd[31:16] := Rn[31:16] - Rm[31:16], Rd[15:0] := Rn[15:0] - Rm[15:0]	G
	Byte-wise addition	6	<prefix>ADD8 Rd, Rn, Rm		Rd[31:24] := Rn[31:24] + Rm[31:24], Rd[23:16] := Rn[23:16] + Rm[23:16], Rd[15:8] := Rn[15:8] + Rm[15:8], Rd[7:0] := Rn[7:0] + Rm[7:0]	G
	Byte-wise subtraction	6	<prefix>SUB8 Rd, Rn, Rm		Rd[31:24] := Rn[31:24] - Rm[31:24], Rd[23:16] := Rn[23:16] - Rm[23:16], Rd[15:8] := Rn[15:8] - Rm[15:8], Rd[7:0] := Rn[7:0] - Rm[7:0]	G
	Halfword-wise exchange, add, subtract	6	<prefix>ASX Rd, Rn, Rm		Rd[31:16] := Rn[31:16] + Rm[15:0], Rd[15:0] := Rn[15:0] - Rm[31:16]	G
	Halfword-wise exchange, subtract, add	6	<prefix>SAX Rd, Rn, Rm		Rd[31:16] := Rn[31:16] - Rm[15:0], Rd[15:0] := Rn[15:0] + Rm[31:16]	G
	Unsigned sum of absolute differences	6	USAD8 Rd, Rm, Rs		Rd := Abs(Rm[31:24] - Rs[31:24]) + Abs(Rm[23:16] - Rs[23:16]) + Abs(Rm[15:8] - Rs[15:8]) + Abs(Rm[7:0] - Rs[7:0])	
	and accumulate	6	USADA8 Rd, Rm, Rs, Rn		Rd := Rn + Abs(Rm[31:24] - Rs[31:24]) + Abs(Rm[23:16] - Rs[23:16]) + Abs(Rm[15:8] - Rs[15:8]) + Abs(Rm[7:0] - Rs[7:0])	
Saturate	Signed saturate word, right shift	6	SSAT Rd, #<sat>, Rm{, ASR <sh>}		Rd := SignedSat((Rm ASR sh), sat). <sat> range 1-32, <sh> range 1-31.	Q, R
	Signed saturate word, left shift	6	SSAT Rd, #<sat>, Rm{, LSL <sh>}		Rd := SignedSat((Rm LSL sh), sat). <sat> range 1-32, <sh> range 0-31.	Q
	Signed saturate two halfwords	6	SSAT16 Rd, #<sat>, Rm		Rd[31:16] := SignedSat(Rm[31:16], sat). Rd[15:0] := SignedSat(Rm[15:0], sat). <sat> range 1-16.	Q
	Unsigned saturate word, right shift	6	USAT Rd, #<sat>, Rm{, ASR <sh>}		Rd := UnsignedSat((Rm ASR sh), sat). <sat> range 0-31, <sh> range 1-31.	Q, R
	Unsigned saturate word, left shift	6	USAT Rd, #<sat>, Rm{, LSL <sh>}		Rd := UnsignedSat((Rm LSL sh), sat). <sat> range 0-31, <sh> range 0-31.	Q
	Unsigned saturate two halfwords	6	USAT16 Rd, #<sat>, Rm		Rd[31:16] := UnsignedSat(Rm[31:16], sat). Rd[15:0] := UnsignedSat(Rm[15:0], sat). <sat> range 0-15.	Q

ARM and Thumb-2 Instruction Set Quick Reference Card

Operation		§	Assembler	S updates	Action	Notes
Multiply	Multiply		MUL{S} Rd, Rm, Rs	N Z C*	Rd := (Rm * Rs)[31:0] (If Rm is Rd, S can be used in Thumb-2)	N, S
	and accumulate		MLA{S} Rd, Rm, Rs, Rn	N Z C*	Rd := (Rn + (Rm * Rs))[31:0]	S
	and subtract	T2	MLS Rd, Rm, Rs, Rn		Rd := (Rn – (Rm * Rs))[31:0]	
	unsigned long		UMULL{S} RdLo, RdHi, Rm, Rs	N Z C* V*	RdHi, RdLo := unsigned(Rm * Rs)	S
	unsigned accumulate long		UMLAL{S} RdLo, RdHi, Rm, Rs	N Z C* V*	RdHi, RdLo := unsigned(RdHi, RdLo + Rm * Rs)	S
	unsigned double accumulate long	6	UMAAL RdLo, RdHi, Rm, Rs		RdHi, RdLo := unsigned(RdHi + RdLo + Rm * Rs)	
	Signed multiply long		SMULL{S} RdLo, RdHi, Rm, Rs	N Z C* V*	RdHi, RdLo := signed(Rm * Rs)	S
	and accumulate long		SMLAL{S} RdLo, RdHi, Rm, Rs	N Z C* V*	RdHi, RdLo := signed(RdHi, RdLo + Rm * Rs)	S
	16 * 16 bit	5E	SMULxy Rd, Rm, Rs		Rd := Rm[x] * Rs[y]	
	32 * 16 bit	5E	SMULWy Rd, Rm, Rs		Rd := (Rm * Rs[y])[47:16]	
	16 * 16 bit and accumulate	5E	SMLAxy Rd, Rm, Rs, Rn		Rd := Rn + Rm[x] * Rs[y]	Q
	32 * 16 bit and accumulate	5E	SMLAWy Rd, Rm, Rs, Rn		Rd := Rn + (Rm * Rs[y])[47:16]	Q
	16 * 16 bit and accumulate long	5E	SMLALxy RdLo, RdHi, Rm, Rs		RdHi, RdLo := RdHi, RdLo + Rm[x] * Rs[y]	
	Dual signed multiply, add	6	SMUAD{X} Rd, Rm, Rs		Rd := Rm[15:0] * RsX[15:0] + Rm[31:16] * RsX[31:16]	Q
	and accumulate	6	SMLAD{X} Rd, Rm, Rs, Rn		Rd := Rn + Rm[15:0] * RsX[15:0] + Rm[31:16] * RsX[31:16]	Q
	and accumulate long	6	SMLALD{X} RdLo, RdHi, Rm, Rs		RdHi, RdLo := RdHi, RdLo + Rm[15:0] * RsX[15:0] + Rm[31:16] * RsX[31:16]	
	Dual signed multiply, subtract	6	SMUSD{X} Rd, Rm, Rs		Rd := Rm[15:0] * RsX[15:0] – Rm[31:16] * RsX[31:16]	Q
	and accumulate	6	SMLSD{X} Rd, Rm, Rs, Rn		Rd := Rn + Rm[15:0] * RsX[15:0] – Rm[31:16] * RsX[31:16]	Q
	and accumulate long	6	SMLSLD{X} RdLo, RdHi, Rm, Rs		RdHi, RdLo := RdHi, RdLo + Rm[15:0] * RsX[15:0] – Rm[31:16] * RsX[31:16]	
	Signed top word multiply	6	SMMUL{R} Rd, Rm, Rs		Rd := (Rm * Rs)[63:32]	
	and accumulate	6	SMMLA{R} Rd, Rm, Rs, Rn		Rd := Rn + (Rm * Rs)[63:32]	
	and subtract	6	SMMLS{R} Rd, Rm, Rs, Rn		Rd := Rn – (Rm * Rs)[63:32]	
	with internal 40-bit accumulate	XS	MIA Ac, Rm, Rs		Ac := Ac + Rm * Rs	
	packed halfword	XS	MIAPH Ac, Rm, Rs		Ac := Ac + Rm[15:0] * Rs[15:0] + Rm[31:16] * Rs[31:16]	
	halfword	XS	MIAXy Ac, Rm, Rs		Ac := Ac + Rm[x] * Rs[y]	
Divide	Signed or Unsigned	RM	<op> Rd, Rn, Rm		Rd := Rn / Rm <op> is SDIV (signed) or UDIV (unsigned)	
Move data	Move		MOV{S} Rd, <Operand2>	N Z C	Rd := Operand2 See also Shift instructions	N
	NOT		MVN{S} Rd, <Operand2>	N Z C	Rd := 0xFFFFFFFF EOR Operand2	N
	top	T2	MOVT Rd, #<imm16>		Rd[31:16] := imm16, Rd[15:0] unaffected, imm16 range 0-65535	
	wide	T2	MOV Rd, #<imm16>		Rd[15:0] := imm16, Rd[31:16] = 0, imm16 range 0-65535	
	40-bit accumulator to register	XS	MRA RdLo, RdHi, Ac		RdLo := Ac[31:0], RdHi := Ac[39:32]	
Shift	register to 40-bit accumulator	XS	MAR Ac, RdLo, RdHi		Ac[31:0] := RdLo, Ac[39:32] := RdHi	
	Arithmetic shift right		ASR{S} Rd, Rm, <Rs sh>	N Z C	Rd := ASR(Rm, Rslsh) Same as MOV{S} Rd, Rm, ASR <Rs sh>	N
	Logical shift left		LSL{S} Rd, Rm, <Rs sh>	N Z C	Rd := LSL(Rm, Rslsh) Same as MOV{S} Rd, Rm, LSL <Rs sh>	N
	Logical shift right		LSR{S} Rd, Rm, <Rs sh>	N Z C	Rd := LSR(Rm, Rslsh) Same as MOV{S} Rd, Rm, LSR <Rs sh>	N
	Rotate right		ROR{S} Rd, Rm, <Rs sh>	N Z C	Rd := ROR(Rm, Rslsh) Same as MOV{S} Rd, Rm, ROR <Rs sh>	N
	Rotate right with extend		RRX{S} Rd, Rm	N Z C	Rd := RRX(Rm) Same as MOV{S} Rd, Rm, RRX	
Count leading zeros		5	CLZ Rd, Rm		Rd := number of leading zeros in Rm	
Compare	Compare		CMP Rn, <Operand2>	N Z C V	Update CPSR flags on Rn – Operand2	N
	negative		CMN Rn, <Operand2>	N Z C V	Update CPSR flags on Rn + Operand2	N
Logical	Test		TST Rn, <Operand2>	N Z C	Update CPSR flags on Rn AND Operand2	N
	Test equivalence		TEQ Rn, <Operand2>	N Z C	Update CPSR flags on Rn EOR Operand2	
	AND		AND{S} Rd, Rn, <Operand2>	N Z C	Rd := Rn AND Operand2	N
	EOR		EOR{S} Rd, Rn, <Operand2>	N Z C	Rd := Rn EOR Operand2	N
	ORR		ORR{S} Rd, Rn, <Operand2>	N Z C	Rd := Rn OR Operand2	N
	ORN	T2	ORN{S} Rd, Rn, <Operand2>	N Z C	Rd := Rn OR NOT Operand2	T
	Bit Clear		BIC{S} Rd, Rn, <Operand2>	N Z C	Rd := Rn AND NOT Operand2	N

ARM and Thumb-2 Instruction Set

Quick Reference Card

Operation		§	Assembler	Action	Notes
Bit field	Bit Field Clear	T2	BFC Rd, #<lsb>, #<width>	Rd[(width+lsb-1):lsb] := 0, other bits of Rd unaffected	
	Bit Field Insert	T2	BFI Rd, Rn, #<lsb>, #<width>	Rd[(width+lsb-1):lsb] := Rn[(width-1):0], other bits of Rd unaffected	
	Signed Bit Field Extract	T2	SBFX Rd, Rn, #<lsb>, #<width>	Rd[(width-1):0] = Rn[(width+lsb-1):lsb], Rd[31:width] = Replicate(Rn[width+lsb-1])	
	Unsigned Bit Field Extract	T2	UBFX Rd, Rn, #<lsb>, #<width>	Rd[(width-1):0] = Rn[(width+lsb-1):lsb], Rd[31:width] = Replicate(0)	
Pack	Pack halfword bottom + top	6	PKHBT Rd, Rn, Rm{, LSL #<sh>}	Rd[15:0] := Rn[15:0], Rd[31:16] := (Rm LSL sh)[31:16]. sh 0-31.	
	Pack halfword top + bottom	6	PKHTB Rd, Rn, Rm{, ASR #<sh>}	Rd[31:16] := Rn[31:16], Rd[15:0] := (Rm ASR sh)[15:0]. sh 1-32.	
Signed extend	Halfword to word	6	SXTH Rd, Rm{, ROR #<sh>}	Rd[31:0] := SignExtend((Rm ROR (8 * sh))[15:0]). sh 0-3.	N
	Two bytes to halfwords	6	SXTB16 Rd, Rm{, ROR #<sh>}	Rd[31:16] := SignExtend((Rm ROR (8 * sh))[23:16]). Rd[15:0] := SignExtend((Rm ROR (8 * sh))[7:0]). sh 0-3.	
	Byte to word	6	SXTB Rd, Rm{, ROR #<sh>}	Rd[31:0] := SignExtend((Rm ROR (8 * sh))[7:0]). sh 0-3.	N
Unsigned extend	Halfword to word	6	UXTH Rd, Rm{, ROR #<sh>}	Rd[31:0] := ZeroExtend((Rm ROR (8 * sh))[15:0]). sh 0-3.	N
	Two bytes to halfwords	6	UXTB16 Rd, Rm{, ROR #<sh>}	Rd[31:16] := ZeroExtend((Rm ROR (8 * sh))[23:16]). Rd[15:0] := ZeroExtend((Rm ROR (8 * sh))[7:0]). sh 0-3.	
	Byte to word	6	UXTB Rd, Rm{, ROR #<sh>}	Rd[31:0] := ZeroExtend((Rm ROR (8 * sh))[7:0]). sh 0-3.	N
Signed extend with add	Halfword to word, add	6	SXTAH Rd, Rn, Rm{, ROR #<sh>}	Rd[31:0] := Rn[31:0] + SignExtend((Rm ROR (8 * sh))[15:0]). sh 0-3.	
	Two bytes to halfwords, add	6	SXTAB16 Rd, Rn, Rm{, ROR #<sh>}	Rd[31:16] := Rn[31:16] + SignExtend((Rm ROR (8 * sh))[23:16]), Rd[15:0] := Rn[15:0] + SignExtend((Rm ROR (8 * sh))[7:0]). sh 0-3.	
	Byte to word, add	6	SXTAB Rd, Rn, Rm{, ROR #<sh>}	Rd[31:0] := Rn[31:0] + SignExtend((Rm ROR (8 * sh))[7:0]). sh 0-3.	
Unsigned extend with add	Halfword to word, add	6	UXTAH Rd, Rn, Rm{, ROR #<sh>}	Rd[31:0] := Rn[31:0] + ZeroExtend((Rm ROR (8 * sh))[15:0]). sh 0-3.	
	Two bytes to halfwords, add	6	UXTAB16 Rd, Rn, Rm{, ROR #<sh>}	Rd[31:16] := Rn[31:16] + ZeroExtend((Rm ROR (8 * sh))[23:16]), Rd[15:0] := Rn[15:0] + ZeroExtend((Rm ROR (8 * sh))[7:0]). sh 0-3.	
	Byte to word, add	6	UXTAB Rd, Rn, Rm{, ROR #<sh>}	Rd[31:0] := Rn[31:0] + ZeroExtend((Rm ROR (8 * sh))[7:0]). sh 0-3.	
Reverse	Bits in word	T2	RBIT Rd, Rm	For (i = 0; i < 32; i++) : Rd[i] = Rm[31-i]	
	Bytes in word	6	REV Rd, Rm	Rd[31:24] := Rm[7:0], Rd[23:16] := Rm[15:8], Rd[15:8] := Rm[23:16], Rd[7:0] := Rm[31:24]	N
	Bytes in both halfwords	6	REV16 Rd, Rm	Rd[15:8] := Rm[7:0], Rd[7:0] := Rm[15:8], Rd[31:24] := Rm[23:16], Rd[23:16] := Rm[31:24]	N
	Bytes in low halfword, sign extend	6	REVSH Rd, Rm	Rd[15:8] := Rm[7:0], Rd[7:0] := Rm[15:8], Rd[31:16] := Rm[7] * &FFFF	N
Select	Select bytes	6	SEL Rd, Rn, Rm	Rd[7:0] := Rn[7:0] if GE[0] = 1, else Rd[7:0] := Rm[7:0] Bits[15:8], [23:16], [31:24] selected similarly by GE[1], GE[2], GE[3]	
If-Then	If-Then	T2	IT{pattern} {cond}	Makes up to four following instructions conditional, according to pattern. pattern is a string of up to three letters. Each letter can be T (Then) or E (Else). The first instruction after IT has condition cond. The following instructions have condition cond if the corresponding letter is T, or the inverse of cond if the corresponding letter is E. See Table Condition Field for available condition codes.	T U
Branch	Branch		B <label>	PC := label. label is this instruction ±32MB (T2: ±16MB, T: -252 - +256B)	N, B
	with link		BL <label>	LR := address of next instruction. PC := label. label is this instruction ±32MB (T2: ±16MB).	
	and exchange	4T	BX Rm	PC := Rm. Target is Thumb if Rm[0] is 1, ARM if Rm[0] is 0.	N
	with link and exchange (1)	5T	BLX <label>	LR := address of next instruction. PC := label. Change instruction set. label is this instruction ±32MB (T2: ±16MB).	C
	with link and exchange (2)	5	BLX Rm	LR := address of next instruction. PC := Rm[31:1]. Change to Thumb if Rm[0] is 1, to ARM if Rm[0] is 0.	N
	and change to Jazelle state	5J	BXJ Rm	Change to Jazelle state if available	
	Compare, branch if (non) zero	T2	CB{N}Z Rn, <label>	If Rn {== or !=} 0 then PC := label. label is (this instruction + 4-130).	N T U
Processor state change	Table Branch Byte	T2	TBB [Rn, Rm]	PC = PC + ZeroExtend(Memory(Rn + Rm, 1) << 1). Branch range 4-512. Rn can be PC.	T U
	Table Branch Halfword	T2	TBH [Rn, Rm, LSL #1]	PC = PC + ZeroExtend(Memory(Rn + Rm << 1, 2) << 1). Branch range 4-131072. Rn can be PC.	T U
Move to or from PSR	PSR to register		MRS Rd, <PSR>	Rd := PSR	
	register to PSR		MSR <PSR>_<fields>, Rm	PSR := Rm (selected bytes only)	
	immediate to PSR		MSR <PSR>_<fields>, #<imm8m>	PSR := imm8_r (selected bytes only)	
Processor state change	Change processor state	6	CPSID <iflags> {, #<p_mode>}	Disable specified interrupts, optional change mode.	U, N
		6	CPSIE <iflags> {, #<p_mode>}	Enable specified interrupts, optional change mode.	U, N
	Change processor mode	6	CPS #<p_mode>		U
	Set endianness	6	SETEND <endianness>	Sets endianness for loads and saves. <endianness> can be BE (Big Endian) or LE (Little Endian).	U, N

ARM Instruction Set Quick Reference Card

Single data item loads and stores		§	Assembler	Action if <op> is LDR	Action if <op> is STR	Notes
Load or store word, byte or halfword	Immediate offset		<op>{size}{T} Rd, [Rn {, #<offset>}]{!}	Rd := [address, size]	[address, size] := Rd	1, N
	Post-indexed, immediate		<op>{size}{T} Rd, [Rn], #<offset>	Rd := [address, size]	[address, size] := Rd	2
	Register offset		<op>{size} Rd, [Rn, +/-Rm {, <opsh>}]{!}	Rd := [address, size]	[address, size] := Rd	3, N
	Post-indexed, register		<op>{size}{T} Rd, [Rn], +/-Rm {, <opsh>}	Rd := [address, size]	[address, size] := Rd	4
	PC-relative		<op>{size} Rd, <label>	Rd := [label, size]	Not available	5, N
Load or store doubleword	Immediate offset	5E	<op>D Rd1, Rd2, [Rn {, #<offset>}]{!}	Rd1 := [address], Rd2 := [address + 4]	[address] := Rd1, [address + 4] := Rd2	6, 9
	Post-indexed, immediate	5E	<op>D Rd1, Rd2, [Rn], #<offset>	Rd1 := [address], Rd2 := [address + 4]	[address] := Rd1, [address + 4] := Rd2	6, 9
	Register offset	5E	<op>D Rd1, Rd2, [Rn, +/-Rm {, <opsh>}]{!}	Rd1 := [address], Rd2 := [address + 4]	[address] := Rd1, [address + 4] := Rd2	7, 9
	Post-indexed, register	5E	<op>D Rd1, Rd2, [Rn], +/-Rm {, <opsh>}	Rd1 := [address], Rd2 := [address + 4]	[address] := Rd1, [address + 4] := Rd2	7, 9
	PC-relative	5E	<op>D Rd1, Rd2, <label>	Rd1 := [label], Rd2 := [label + 4]	Not available	8, 9

Preload data or instruction		§ (PLD)	§ (PLI)	Assembler	Action if <op> is PLD	Action if <op> is PLI	Notes
	Immediate offset	5E	7	<op> [Rn {, #<offset>}]	Preload [address, 32] (data)	Preload [address, 32] (instruction)	1, C
	Register offset	5E	7	<op> [Rn, +/-Rm {, <opsh>}]	Preload [address, 32] (data)	Preload [address, 32] (instruction)	3, C
	PC-relative	5E	7	<op> <label>	Preload [label, 32] (data)	Preload [label, 32] (instruction)	5, C

Other memory operations		§	Assembler	Action	Notes
Load multiple	Block data load		LDM{IA IB DA DB} Rn{!}, <reglist-PC>	Load list of registers from [Rn]	N, I
	return (and exchange) and restore CPSR		LDM{IA IB DA DB} Rn{!}, <reglist+PC>	Load registers, PC := [address][31:1] (§ 5T: Change to Thumb if [address][0] is 1)	I
	User mode registers		LDM{IA IB DA DB} Rn{!}, <reglist+PC>^	Load registers, branch (§ 5T: and exchange), CPSR := SPSR. Exception modes only.	I
			LDM{IA IB DA DB} Rn, <reglist-PC>^	Load list of User mode registers from [Rn]. Privileged modes only.	I
Pop			POP <reglist>	Canonical form of LDM SP!, <reglist>	N
Load exclusive	Semaphore operation	6	LDREX Rd, [Rn]	Rd := [Rn], tag address as exclusive access. Outstanding tag set if not shared address. Rd, Rn not PC.	
	Halfword or Byte	6K	LDREX{H B} Rd, [Rn]	Rd[15:0] := [Rn] or Rd[7:0] := [Rn], tag address as exclusive access. Outstanding tag set if not shared address. Rd, Rn not PC.	
	Doubleword	6K	LDREXD Rd1, Rd2, [Rn]	Rd1 := [Rn], Rd2 := [Rn+4], tag addresses as exclusive access. Outstanding tags set if not shared addresses. Rd1, Rd2, Rn not PC.	9
Store multiple	Push, or Block data store		STM{IA IB DA DB} Rn{!}, <reglist>	Store list of registers to [Rn]	N, I
	User mode registers		STM{IA IB DA DB} Rn{!}, <reglist>^	Store list of User mode registers to [Rn]. Privileged modes only.	I
Push			PUSH <reglist>	Canonical form of STMDB SP!, <reglist>	N
Store exclusive	Semaphore operation	6	STREX Rd, Rm, [Rn]	If allowed, [Rn] := Rm, clear exclusive tag, Rd := 0. Else Rd := 1. Rd, Rm, Rn not PC.	
	Halfword or Byte	6K	STREX{H B} Rd, Rm, [Rn]	If allowed, [Rn] := Rm[15:0] or [Rn] := Rm[7:0], clear exclusive tag, Rd := 0. Else Rd := 1. Rd, Rm, Rn not PC.	
	Doubleword	6K	STREXD Rd, Rm1, Rm2, [Rn]	If allowed, [Rn] := Rm1, [Rn+4] := Rm2, clear exclusive tags, Rd := 0. Else Rd := 1. Rd, Rm1, Rm2, Rn not PC.	9
Clear exclusive		6K	CLREX	Clear local processor exclusive tag	C

Notes: availability and range of options for Load, Store, and Preload operations					
Note	ARM Word, B, D	ARM SB, H, SH	ARM T, BT	Thumb-2 Word, B, SB, H, SH, D	Thumb-2 T, BT, SBT, HT, SHT
1	offset: –4095 to +4095	offset: –255 to +255	Not available	offset: –255 to +255 if writeback, –255 to +4095 otherwise	offset: 0 to +255, writeback not allowed
2	offset: –4095 to +4095	offset: –255 to +255	offset: –4095 to +4095	offset: –255 to +255	Not available
3	Full range of {, <opsh>}	{, <opsh>} not allowed	Not available	<opsh> restricted to LSL #<sh>, <sh> range 0 to 3	Not available
4	Full range of {, <opsh>}	{, <opsh>} not allowed	Full range of {, <opsh>}	Not available	Not available
5	label within +/- 4092 of current instruction	Not available	Not available	label within +/- 4092 of current instruction	Not available
6	offset: –255 to +255	-	-	offset: –1020 to +1020, must be multiple of 4.	-
7	{, <opsh>} not allowed	-	-	Not available	-
8	label within +/- 252 of current instruction	-	-	Not available	-
9	Rd1 even, and not r14, Rd2 == Rd1 + 1.	-	-	Rd1 != PC, Rd2 != PC	-

ARM Instruction Set Quick Reference Card

Coprocessor operations	§	Assembler	Action	Notes
Data operations		CDP{2} <copr>, <op1>, CRd, CRn, CRm{, <op2>}	Coprocessor defined	C2
Move to ARM register from coprocessor		MRC{2} <copr>, <op1>, Rd, CRn, CRm{, <op2>}	Coprocessor defined	C2
Two ARM register move	5E	MRRC <copr>, <op1>, Rd, Rn, CRm	Coprocessor defined	C
Alternative two ARM register move	6	MRRC2 <copr>, <op1>, Rd, Rn, CRm	Coprocessor defined	C2
Move to coproc from ARM reg		MCR{2} <copr>, <op1>, Rd, CRn, CRm{, <op2>}	Coprocessor defined	C2
Two ARM register move	5E	MCRR <copr>, <op1>, Rd, Rn, CRm	Coprocessor defined	C
Alternative two ARM register move	6	MCRR2 <copr>, <op1>, Rd, Rn, CRm	Coprocessor defined	C2
Loads and stores, pre-indexed		<op>{2} <copr>, CRd, [Rn, #+/-<offset8*4>]{!}	op: LDC or STC. offset: multiple of 4 in range 0 to 1020.	C2
Loads and stores, zero offset		<op>{2} <copr>, CRd, [Rn] {, 8-bit copro. option}	op: LDC or STC.	C2
Loads and stores, post-indexed		<op>{2} <copr>, CRd, [Rn], #+/-<offset8*4>	op: LDC or STC. offset: multiple of 4 in range 0 to 1020.	C2

Miscellaneous operations	§	Assembler	Action	Notes
Swap word		SWP Rd, Rm, [Rn]	temp := [Rn], [Rn] := Rm, Rd := temp.	D
Swap byte		SWPB Rd, Rm, [Rn]	temp := ZeroExtend([Rn][7:0]), [Rn][7:0] := Rm[7:0], Rd := temp	D
Store return state	6	SRS{IA IB DA DB} SP{!}, #<p_mode>	[SPm] := LR, [SPm + 4] := CPSR	C, I
Return from exception	6	RFE{IA IB DA DB} Rn{!}	PC := [Rn], CPSR := [Rn + 4]	C, I
Breakpoint	5	BKPT <imm16>	Prefetch abort <i>or</i> enter debug state. 16-bit bitfield encoded in instruction.	C, N
Secure Monitor Call	Z	SMC <imm16>	Secure Monitor Call exception. 16-bit bitfield encoded in instruction. Formerly SMI.	N
Supervisor Call		SVC <imm24>	Supervisor Call exception. 24-bit bitfield encoded in instruction. Formerly SWI.	N
No operation	6	NOP	None, might not even consume any time.	N
Hints				
Debug Hint	7	DBG	Provide hint to debug and related systems.	
Data Memory Barrier	7	DMB	Ensure the order of observation of memory accesses.	C
Data Synchronization Barrier	7	DSB	Ensure the completion of memory accesses.	C
Instruction Synchronization Barrier	7	ISB	Flush processor pipeline and branch prediction logic.	C
Set event	T2	SEV	Signal event in multiprocessor system. NOP if not implemented.	N
Wait for event	T2	WFE	Wait for event, IRQ, FIQ, Imprecise abort, or Debug entry request. NOP if not implemented.	N
Wait for interrupt	T2	WFI	Wait for IRQ, FIQ, Imprecise abort, or Debug entry request. NOP if not implemented.	N
Yield	T2	YIELD	Yield control to alternative thread. NOP if not implemented.	N

Notes				
A	Not available in Thumb state.	N	Some or all forms of this instruction are 16-bit (Narrow) instructions in Thumb-2 code. For details see the <i>Thumb 16-bit Instruction Set (UAL) Quick Reference Card</i> .	
B	Can be conditional in Thumb state without having to be in an IT block.	P	Rn can be the PC in Thumb state in this instruction.	
C	Condition codes are not allowed in ARM state.	Q	Sets the Q flag if saturation (addition or subtraction) or overflow (multiplication) occurs. Read and reset the Q flag using MRS and MSR.	
C2	The optional 2 is available from ARMv5. It provides an alternative operation. Condition codes are not allowed for the alternative form in ARM state.	R	<sh> range is 1-32 in the ARM instruction.	
D	Deprecated. Use LDREX and STREX instead.	S	The S modifier is not available in the Thumb-2 instruction.	
G	Updates the four GE flags in the CPSR based on the results of the individual operations.	T	Not available in ARM state.	
I	IA is the default, and is normally omitted.	U	Not allowed in an IT block. Condition codes not allowed in either ARM or Thumb state.	
L	ARM: <imm8m>, 16-bit Thumb: multiple of 4 in range 0-1020. 32-bit Thumb: 0-4095.			

ARM Instruction Set
Quick Reference Card

ARM architecture versions	
<i>n</i>	ARM architecture version <i>n</i> and above
<i>n</i> T, <i>n</i> J	T or J variants of ARM architecture version <i>n</i> and above
5E	ARM v5E, and 6 and above
T2	All Thumb-2 versions of ARM v6 and above
6K	ARMv6K and above for ARM instructions, ARMv7 for Thumb
Z	All Security extension versions of ARMv6 and above
RM	ARMv7-R and ARMv7-M only
XS	XScale coprocessor instruction

Flexible Operand 2		
Immediate value		#<imm8m>
Register, optionally shifted by constant (see below)		Rm {, <opsh>}
Register, logical shift left by register		Rm, LSL Rs
Register, logical shift right by register		Rm, LSR Rs
Register, arithmetic shift right by register		Rm, ASR Rs
Register, rotate right by register		Rm, ROR Rs

Register, optionally shifted by constant		
(No shift)	Rm	Same as Rm, LSL #0
Logical shift left	Rm, LSL #<shift>	Allowed shifts 0-31
Logical shift right	Rm, LSR #<shift>	Allowed shifts 1-32
Arithmetic shift right	Rm, ASR #<shift>	Allowed shifts 1-32
Rotate right	Rm, ROR #<shift>	Allowed shifts 1-31
Rotate right with extend	Rm, RRX	

PSR fields (use at least one suffix)		
Suffix	Meaning	
c	Control field mask byte	PSR[7:0]
f	Flags field mask byte	PSR[31:24]
s	Status field mask byte	PSR[23:16]
x	Extension field mask byte	PSR[15:8]

Condition Field		
Mnemonic	Description	Description (VFP)
EQ	Equal	Equal
NE	Not equal	Not equal, or unordered
CS / HS	Carry Set / Unsigned higher or same	Greater than or equal, or unordered
CC / LO	Carry Clear / Unsigned lower	Less than
MI	Negative	Less than
PL	Positive or zero	Greater than or equal, or unordered
VS	Overflow	Unordered (at least one NaN operand)
VC	No overflow	Not unordered
HI	Unsigned higher	Greater than, or unordered
LS	Unsigned lower or same	Less than or equal
GE	Signed greater than or equal	Greater than or equal
LT	Signed less than	Less than, or unordered
GT	Signed greater than	Greater than
LE	Signed less than or equal	Less than or equal, or unordered
AL	Always (normally omitted)	Always (normally omitted)
All ARM instructions (except those with Note C or Note U) can have any one of these condition codes after the instruction mnemonic (that is, before the first space in the instruction as shown on this card). This condition is encoded in the instruction.		
All Thumb-2 instructions (except those with Note U) can have any one of these condition codes after the instruction mnemonic. This condition is encoded in a preceding IT instruction (except in the case of conditional Branch instructions). Condition codes in instructions must match those in the preceding IT instruction.		
On processors without Thumb-2, the only Thumb instruction that can have a condition code is B <label>.		

Processor Modes	
16	User
17	FIQ Fast Interrupt
18	IRQ Interrupt
19	Supervisor
23	Abort
27	Undefined
31	System

Prefixes for Parallel Instructions	
S	Signed arithmetic modulo 2 ⁸ or 2 ¹⁶ , sets CPSR GE bits
Q	Signed saturating arithmetic
SH	Signed arithmetic, halving results
U	Unsigned arithmetic modulo 2 ⁸ or 2 ¹⁶ , sets CPSR GE bits
UQ	Unsigned saturating arithmetic
UH	Unsigned arithmetic, halving results

Proprietary Notice

Words and logos marked with ® or ™ are registered trademarks or trademarks owned by ARM Limited. Other brands and names mentioned herein may be the trademarks of their respective owners.

Neither the whole nor any part of the information contained in, or the product described in, this document may be adapted or reproduced in any material form except with the prior written permission of the copyright holder.

The product described in this document is subject to continuous developments and improvements. All particulars of the product and its use contained in this document are given by ARM in good faith. However, all warranties implied or expressed, including but not limited to implied warranties of merchantability, or fitness for purpose, are excluded.

This reference card is intended only to assist the reader in the use of the product. ARM Ltd shall not be liable for any loss or damage arising from the use of any information in this reference card, or any error or omission in such information, or any incorrect use of the product.

Document Number

ARM QRC 0001L

Change Log

Issue	Date	Change	Issue	Date	Change
A	June 1995	First Release	B	Sept 1996	Second Release
C	Nov 1998	Third Release	D	Oct 1999	Fourth Release
E	Oct 2000	Fifth Release	F	Sept 2001	Sixth Release
G	Jan 2003	Seventh Release	H	Oct 2003	Eighth Release
I	Dec 2004	Ninth Release	J	May 2005	RVCT 2.2 SP1
K	March 2006	RVCT 3.0	L	March 2007	RVCT 3.1